



Mini Project:

Library Management System

Programme	: Bachelor of Computer Science (<i>Data Engineering</i>)
Subject Code	: SECJ2154
Subject Name	: Object Oriented Programming (OOP)
Session-Sem	: 2024/2025-2

Prepared by	: 1) Joanne Ching Yin Xuan (A23CS0227)
	: 2) Lim Yu Han (A23CS0241)

Section	: 03
----------------	------

Lecturer	: Dr. Muhammad Khatibsyarbini
-----------------	-------------------------------

Table of Contents

1.0	Project Description	3
1.1	System Functions.....	3
2.0	Theory Implementation	4
3.0	Sample Output	6
4.0	UML Class Diagram	8

1.0 Project Description

The Library Management System is designed to help a librarian efficiently manage a collection of books. The system allows the librarian to perform tasks such as adding new books, deleting existing ones, viewing all books, issuing books to users, and handling book returns. It is a console-based Java application that showcases real-world programming by applying object-oriented concepts and Java collection frameworks.

1.1 System Functions

1. Login Authentication

- A login interface allows only authorized librarians to access the system.
- Requires a hardcoded librarian ID and password.

2. Add Book

- The librarian can input a book's ID, title, and author to add it to the library.
- New books are stored dynamically using an ArrayList.

3. Delete Book

- A book can be removed by entering its ID.
- If the book ID is not found, a custom exception is thrown and handled.

4. View All Books

- Displays a list of all books currently stored in the system.
- Each book displays its status (Available or Issued).

5. Issue Book

- Marks a book as issued using its ID.
- If the book is already issued or not found, appropriate messages or exceptions are displayed.

6. Return Book

- Allows the librarian to mark a book as returned.
- Ensures proper validation to avoid errors.

2.0 Theory Implementation

Chapter 5: Vectors and ArrayList

The system uses `ArrayList<Book>` in the `Library` class to store books dynamically. This allows the system to grow as more books are added, and supports built-in methods like `add()`, `remove()`, `get()`, and iteration via `for-each` loop.

```
private ArrayList<Book> books = new ArrayList<>(); // Chapter 5: ArrayList
```

Chapter 6: Class Relationships

The system reflects how real-world entities relate without owning the lifetime of the objects. The system demonstrates:

- Aggregation: The `Library` class aggregates multiple `Book` objects (Library "has-a" collection of Books).

```
public class Library {  
    private ArrayList<Book> books = new ArrayList<>(); // Chapter 5: ArrayList  
    // Chapter 6: Aggregation (Library "has-a" list of Books)
```

- Association: The `Librarian` class interacts with the `Library` class to perform management tasks.

```
public class Librarian extends User {  
    private Library library; // Chapter 6: Association (Librarian uses Library class)
```

Chapter 7: Inheritance

The `User` class is an abstract superclass with common user information. The `Librarian` class inherits from `User` and overrides the `displayMenu()` method. This enables future expansion to different user types, like `Students` or `Guests`.

```
public class Librarian extends User { // Chapter 7: Inheritance (Librarian extends User)  
    private Library library;
```

Chapter 8: Polymorphism

The system uses dynamic method dispatch:

- The displayMenu() method is declared abstract in User and implemented differently in Librarian.

```
// Chapter 8: Polymorphic method to be overridden
public abstract void displayMenu();
```

- The Main class calls user.displayMenu() using a User reference, invoking the actual implementation at runtime.

```
User librarian = new Librarian(name:"Joanne", id, library);
librarian.displayMenu();
```

Chapter 9: Exception Handling

The system handles runtime errors using:

- try-catch blocks for safe user input.

```
try {
    library.issueBook(id);
} catch (BookNotFoundException e) {
    System.out.println(e.getMessage());
}
```

- A custom exception, BookNotFoundException, to handle invalid book IDs.

```
// Chapter 9: Exception Handling (Custom exception)

public class BookNotFoundException extends Exception {
    public BookNotFoundException(String message) {
        super(message);
    }
}
```

3.0 Sample Output

1. Login Authentication

```
=== Welcome to Library Management System ===  
Enter Librarian ID: admin001  
Enter Password: lib123
```

2. Menu

```
=== Librarian Menu ===  
1. Add Book  
2. Delete Book  
3. View All Books  
4. Issue Book  
5. Return Book  
6. Exit
```

3. Add Book

```
Enter choice: 1  
Enter Book ID: 123  
Enter Title: Programming Language  
Enter Author: Johnson Lewis  
Book added.
```

4. View All Books

```
Enter choice: 3  
123 - Programming Language by Johnson Lewis [Available]  
124 - French Language by Florance Jennifer [Available]  
125 - Data Analytics Programming by George Alexdandra [Available]
```

5. Delete Book

-A book can be removed by entering its ID.

```
Enter choice: 2  
Enter Book ID to delete: 123  
Book removed.
```

-If the book ID is not found or has already been deleted, a custom exception is thrown and handled.

```
Enter choice: 2  
Enter Book ID to delete: 123  
Book ID 123 not found.
```

6. Issue Book

-Marks a book as issued using its ID.

```
Enter choice: 4
Enter Book ID to issue: 123
Book issued.
```

-Appropriate exceptions are displayed if the book is already issued or not found.

```
Enter choice: 4
Enter Book ID to issue: 123
Book already issued.
```

```
Enter choice: 4
Enter Book ID to issue: 126
Book ID 126 not found.
```

7. Return Book

-Allows the librarian to mark a book as returned.

```
Enter choice: 5
Enter Book ID to return: 123
Book returned.
```

-Proper validation to avoid errors.

```
Enter choice: 5
Enter Book ID to return: 124
Book is not issued.
```

4.0 UML Class Diagram

